

Garcia, Seane Karl S.

CS-401

2203-6APSI

Design System

The tokens and components below come from the real PixelRack client. Colours and fonts come from the `@theme` block in `client/src/index.css`. Components come from `client/src/components/`. None of this was decided in the abstract. Contrast ratios are computed, not guessed.

Step A: Choose your styling approach

My approach: Tailwind CSS, version 4, with a CSS-first theme. Tokens live in the `@theme` block of `client/src/index.css`. Tailwind compiles this straight into real CSS custom properties. Utility classes like `bg-bg-primary` or `text-accent-blue` read from those properties. Tailwind v4 does not need a `tailwind.config.js` file for this. The theme is declared in CSS instead.

Approach	Where tokens live
CSS Modules / plain CSS	:root custom properties (--color-primary)
Tailwind (this project)	the <code>@theme</code> block in <code>client/src/index.css</code> (Tailwind v4 does not need a <code>tailwind.config.js</code> file)
styled-components	a theme object passed through ThemeProvider
A UI library (MUI, Chakra)	the library's theme object. You override its defaults.

Step B: Colour tokens

Five roles, five tokens. Mapped onto the palette already shipping in `index.css`, plus two supporting tokens the app uses constantly: secondary text and the success state.

DESIGN TOKENS: COLOUR

Core palette. Straight from client/src/index.css, the @theme block.

#0F172A	#1E293B	#38BDF8	#F472B6	#F8FAFC
--color-bg page background code: --color-bg-primary	--color-surface cards, panels, modal body code: --color-bg-container	--color-primary links, buttons, active states code: --color-accent-blue	--color-accent one CTA colour, highlights, errors code: --color-accent-pink	--color-text body text, headings code: --color-text-primary

SUPPORTING TOKENS ALREADY IN CODE (beyond the 5 above)

#94A3B8	#4ADE80
--color-text-secondary captions, labels, footer	--color-success success state (BracketButton, uploads OK)

CONTRAST CHECK (WebAIM formula, 4.5:1 minimum for body text)

- text-primary on bg-primary 17.06:1 PASS
- text-secondary on bg-primary 6.96:1 PASS
- text-primary on bg-surface 13.98:1 PASS
- accent-blue link on bg-primary 8.33:1 PASS
- white on Button primary fill (sky-500) 2.77:1 FAIL

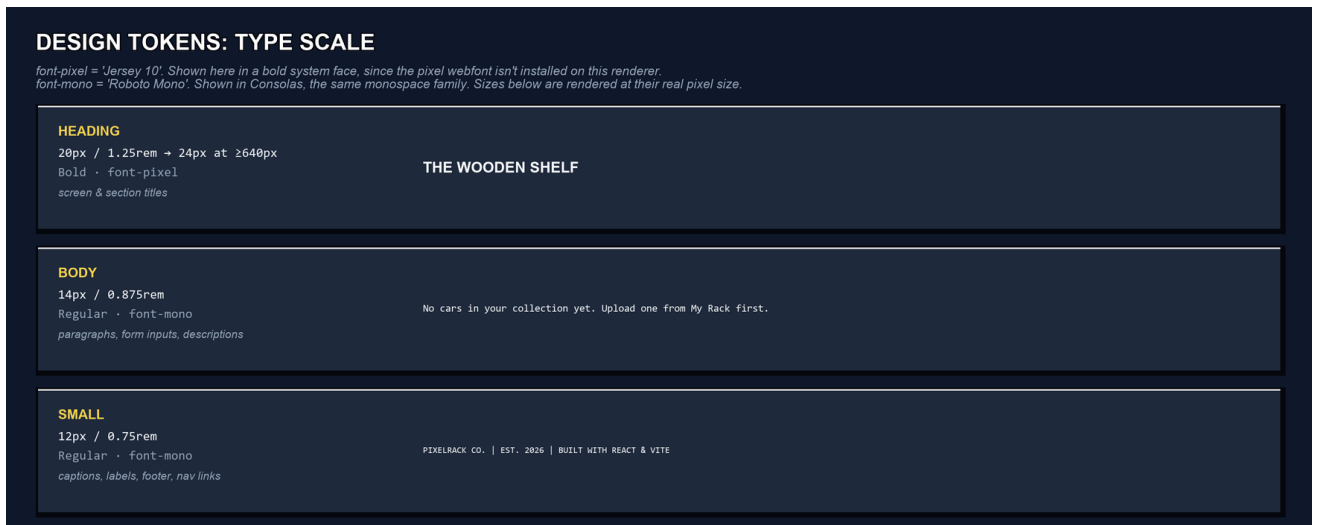
White button labels pass only because of the pixel-text black outline (a text-shadow in every direction). Flat white on the fill alone reads 2.8:1 and would fail on its own.

Colour tokens with hex codes, plus the WebAIM contrast check for every text and background pair actually used.

One honest gap: white button labels sit directly on saturated fills (**sky-500**, **green-500**, **orange-500**). Those read below 3:1 on their own. The **.pixel-text** utility adds a black text-shadow in all four directions, and that is what keeps the labels readable. It is a deliberate fix, not a clean WCAG pass. Rule going forward: **never put flat white text on a saturated button fill without the pixel-text outline.**

Step C: Type scale

Three sizes, rendered below at their real pixel size. font-pixel is shown in a bold system face, and font-mono is shown in Consolas, since the project's webfonts are not installed in this renderer. The sizes and weights themselves are exact.

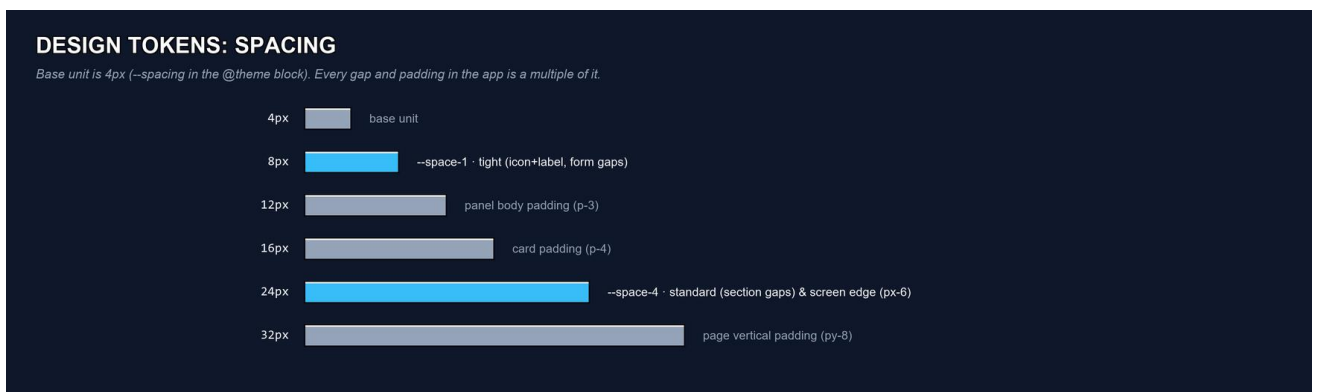


Type scale: Heading (font-pixel 'Jersey 10'), Body and Small (font-mono 'Roboto Mono').

Step D: Spacing rule

Base unit: **4px**. This matches **--spacing: 4px** in the @theme block. It was chosen to line up with the pixel-art grid, not the more common 8px.

- Tight spacing, between related items: **8px** (**--space-1**, Tailwind **gap-2**).
Used for icon and label pairs, the RackHeader stat chip, and form field gaps.
- Standard spacing, between sections: **24px** (**--space-4**, Tailwind **gap-6**).
Used between the Dashboard sidebar and the rack, and between the rack panel and the environment gallery.
- Screen edge padding: **24px** (**px-6**). Used on the Navbar and the Dashboard main column.



Spacing scale drawn to true relative width, from 4px to 32px, with the tight and standard steps highlighted.

Step E: Reusable components

Pulled from the actual component tree. Every entry below renders on more than one screen, or more than once on the same screen. No UI library is in use, so nothing here is "free". Each atom is hand-built in Tailwind.

REUSABLE COMPONENTS

From client/src/components. Every one of these renders on more than one screen.

PRIMARY

SUCCESS

DANGER

Button

ATOM

APPEARS ON

Login, Register, UploadPanel, CarPickerModal, Landing

PROPS

variant, className, disabled, children, ...props

PANEL TITLE

body content rendered here

Panel

MOLECULE

APPEARS ON

UploadPanel, EnvironmentGallery

PROPS

id, title, action, children, bodyClassName, className

HOME MY RACK PROFILE

Navbar

ORGANISM

APPEARS ON

Dashboard, Garage, Konbini

PROPS

none. Reads AuthContext and the current route.

CHOOSE A CAR

CarPickerModal

ORGANISM (overlay)

APPEARS ON

Garage, Konbini (opened from ScenePage, not its own route)

PROPS

cars, placedCarIds, onPick, onClear, onClose, canClear

PIXELRACK CO. | EST. 2026 | BUILT WITH REACT & VITE

Footer

ORGANISM

APPEARS ON

Dashboard, Garage, Konbini

PROPS

none

ShelfCarSlot

MOLECULE

APPEARS ON

Rack (once per car, keyed by car.id)

PROPS

car { id, name, series, color, pixelImageUrl }, onDelete

Six components that repeat across PixelRack, with their real prop signatures.

Component	Level	Appears on	Props it takes
Button	atom	Login, Register, UploadPanel, CarPickerModal, Landing	variant, className, disabled, children

Panel	molecule	UploadPanel, EnvironmentGallery	id, title, action, children, bodyClassName
Navbar	organism	Dashboard, Garage, Konbini	none. Reads AuthContext and the route.
ShelfCarSlot	molecule	Rack (once per car)	car, onDelete
CarPickerModal	organism (overlay)	Garage, Konbini, via ScenePage	cars, placedCarIds, onPick, onClear, onClose, canClear
Footer	organism	Dashboard, Garage, Konbini	none

Not tabled: **BracketButton** is a second button atom, styled green with brackets around the label. It only appears once, in **UploadPanel**'s submit button. That does not meet the worksheet's “more than one screen” rule yet, so it is noted here instead of forced into the table.

Step F: Responsive plan

Below **640px**, on phone: the picker grid drops from 3 columns to 2, car sprites shrink from **h-12** to **h-10**, and Panel and RackHeader titles drop from **text-xl** to **text-lg**. Everything already stacks to one column below this width, since the app is single-column by default.

Above **1024px**, on desktop: the Dashboard switches from a stacked layout to **grid-cols-[320px_1fr]**. The Upload sidebar sits beside the rack instead of above it. This is the only place a second breakpoint is needed. Garage and Konbini stay a single responsive scene box at every width.

These are the only two breakpoints used in the codebase: **sm:** at 640px and **lg:** at 1024px. Confirmed by grep, not assumed. No horizontal scroll at 375px: verified, since every layout is **flex-col** or a single grid column below **sm:**.

Accessibility check (before you build)

⚠ **PARTIAL** Every text-on-background pair passes 4.5:1. True for all plain text: 17.1:1, 7.0:1, 14.0:1, and 8.3:1 measured. The three saturated button fills only pass because of the .pixel-text outline, not on flat colour alone. See Step B.

✅ **PASS** Real semantic elements, not a div with an onClick. Navbar uses `<header>` and `<nav>`. DashboardPage uses `<main>`. Footer uses `<footer>`. Every clickable scene slot in ScenePage is a real `<button type="button">`.

✅ **PASS** Every meaningful image has alt text. `CarSprite` defaults to `alt ?? ''`. The Navbar logo has `alt="PixelRack"`. Decorative nav and scene icons use `alt=""`.

❌ **GAP** Every form input has a matching label. **Not yet true.** LoginPage, RegisterPage, and two of the three UploadPanel inputs rely on `placeholder` text alone, with no `<label htmlFor>` and `id` pair. Only the UploadPanel file-drop input is labelled, through its wrapping `<label>`. This is a real to-do before the final project leans on these forms further.

✅ **PASS** Tab-reachable with a visible focus state. All interactive elements are real `<button>`, `<a>`, and `<input>` elements, so Tab order is native. Inputs and selects show `focus:border-accent-blue` explicitly. **Button** and **BracketButton** do not override focus, so they fall back to the browser's default ring. That still works, it just does not match the pixel theme yet.

What to keep

This page, filled in, is the design reference for the rest of the project.

- The colour, type, and spacing tokens above already exist as the `@theme` block in `client/src/index.css`. Nothing needs to migrate. This document just makes them visible in one place.

- Each reusable component in Step E already lives once under `client/src/components/`, and is rendered everywhere it appears. This was confirmed against the real imports, not assumed.
- The responsive plan in Step F maps to the `sm:` and `lg:` breakpoints already in the `classNames`.
- The one open item is the accessibility gap in Step F: add `<label htmlFor>` and `id` pairs to the Login, Register, and UploadPanel text inputs before the final submission.